

Clase Teórica - 28/08 - Estimación Ágil y Gestión de producto

El proceso de estimación se considera vivo porque NO es estático → este proceso lo vamos a ir refinando a medida que vamos avanzando → en principio, como hay tanta incertidumbre, de primeras voy a tratar de combinar técnicas de estimación para minimizar el error de estimación

El error de estimación surge de: no tener la información suficiente, no combinar técnicas, no usar directamente las técnicas, etc.

Nunca se define un valor absoluto, porque siempre va a existir cierta incertidumbre en la estimación

Primero se debe contar → en términos de software, buscamos estimar la cantidad de horas que me va a llevar construir un producto de software → Existen diferentes métodos para contar basado en diferentes cosas (normalmente explotamos en la materia aquellos basados en la experiencia)

Métodos basados en la experiencia → Datos históricos, juicio experto y analogía

- Datos históricos → puede surgir como problema que ningún proyecto es igual a otro → la dificultad es cómo los documentar, comparar y utilizar para estimar
- Juicio de Experto → Puro (un experto, alguien que sabe mucho de gestión de proyectos y quien hace las estimaciones → como problema tiene definir este rol y además cómo validar los conocimientos de este experto. ¿Qué pasa si este experto se va?) y Wideband Delphi (estructura el juicio experto, un conjunto de personas llevan a cabo una estimación grupal buscando un consenso → NO es una votación, sino que con el refinamiento a través de las iteraciones se converge en un único resultado/valor/conclusión → cada uno recibe la información y estima. Estas se ponen en consideración de un coordinador, y busca abrir un debate las estimaciones para poder iterar refinando estas estimaciones hasta converjan en un mismo valor → CONCEPTUALMENTE)

En estimaciones tradicionales → Primero busco estimar el tamaño del producto, luego el esfuerzo para determinar horas lineales de desarrollo y luego se va al calendario para definir costos y recursos → Esto es una introducción conceptual, porque lo vamos a ver la próxima clase.

Estimaciones en Ambientes Ágiles

Sirve de base/herramienta para poder hacer una planificación, de manera que se pueda proteger al equipo y seguir un plan → Tienen una característica distinta sobre las tradicionales (medidas absolutas), mientras que en ambientes ágiles son relativas → Como medida usamos lo que se conoce como Story Point, que solo tiene sentido para el equipo que va a trabajar con ella

Se obtiene una mejor estimación a través de la comparación, como personas somos mejores en esa manera que estimando medidas absolutas

Story Point → Es una medida de tamaño que nos permite darnos de cuenta que tan grande/pequeño es algo → entonces a partir de esto podemos comparar → es el peso que le vamos a dar a cada historia y a partir de este, vamos a poder compararlas → lo definimos a partir de 3 variables; tamaño, esfuerzo y duda.

EL TAMAÑO NO ES ESFUERZO.

Podemos usar diferentes escalas para definir el tamaño → talles de remera, serie de fibonacci, entre otros ejemplos.

Una de las más utilizadas es la serie de fibonacci → La serie de Fibonacci (1, 2, 3, 5, 8, 13, 21...) crece de **forma no lineal** — los saltos se agrandan a medida que los números crecen. Esto obliga al equipo a tomar una decisión real: si algo no encaja claramente en un "5", tenés que elegir entre "3" u "8", no podés quedarte en el medio con un "6" o "7" que no aportan nada. Eso evita perder tiempo discutiendo diferencias que no son perceptibles en la práctica, y refleja honestamente que cuanto más grande la historia, menos precisión tenemos para estimarla.

Poker estimation → las personas que estiman son las que van a llevar a cabo la tarea; los desarrolladores → se utiliza una user story canónica como base de manera que se pueda comparar

Ver el procedimiento Poker Planning → donde se busca que aquellas personas que van a realizar la tarea converjan en una misma conclusión.

Es importante de todo esto que a un conjunto de historias que ya pueden ser estimadas, le podamos asignar un valor relativo estimado, asignándole un puntaje (story point) que represente su tamaño en comparación con las demás historias del backlog.

GESTIÓN DE PRODUCTO

El mayor riesgo que tenemos al construir software, es construir algo que NADIE va a utilizar (desperdicio) → crear mas software y mas rapido NO implica dar valor, el valor esta en la adopcion del producto por el cliente

No solo alcanza con un producto que satisfaga las necesidades → es por ello que trabajamos con una filosofía que se llama Lean Startup .

LEAN STARTUP → La idea es que en vez de planificar todo de antemano y construir el producto completo, se avanza en ciclos o iteraciones relativamente cortas: construyo lo mínimo para probar una hipótesis, mido cómo reacciona el cliente real, aprendo de eso, y decido si sigo, pivoteo o descarto. El MVP es la herramienta que usás en el paso "construir" de ese ciclo.

Producto mínimo MVP → lo que NO queremos tener es desperdicios (retrabajos o features que no sirven), por lo que trabajamos con MVP → no se crea el producto, el objetivo es validar la hipótesis, es decir, si estoy en la hipótesis correcta de valor que al cliente le sirva → NO tiene relacion con la dimension del producto, NO quiere decir pequeño

No es un prototipo que no funciona — tiene que ser algo real, usable, que el cliente pueda experimentar de verdad. Si no funciona, no estás midiendo nada genuino.

Como se construye → NO tiene que parecer un prototipo/maqueta que no funcione, sino como un pedacito de producto cohesivo → de esta manera valido la hipótesis, que se va refinando con las iteraciones → La bueno de esto,

es que puedo descartarlo rapido, el esfuerzo esta en validar el MVP y no en lo que costo construirlo.

Producto Minimo Comerciable MMF → no tiene como objetivo validar la hipotesis, sino es que es el producto minimo comerciable → se puede decir que es un como un primer release del producto → valor comprobado que esta para salir al mercado

Característica Minima Viable → es una pieza mas chica que el MVP, y suele surgir de querer validar alguna característica pequeña para ver que impacto tiene sobre el cliente.

MVP validado (aprendizaje) → contruir MMP / MMR (Lanzamiento) → MMR 2... → MMR n (evolución)

El que lo va a ver, lo tiene que ver como un producto final (ejemplo dropbox que se planteo como un video)

Concepto	Objetivo	Pregunta que responde
MVP (Producto Mínimo Viable)	Validar una hipótesis de valor	¿Esto que pienso que el cliente quiere, realmente lo quiere?
Característica Mínima Viable	Validar el impacto de una funcionalidad puntual	¿Esta feature específica aporta valor dentro de un producto que ya existe?
MMF/MMP (Mínimo Producto/Feature Comerciable)	Sacar al mercado algo con valor ya comprobado	¿Está listo para vender/lanzar de verdad?
MMR (Mínimo Release Comerciable)	Evolucionar el producto en el mercado	¿Cómo sigo iterando una vez que ya lancé?

Experiencia por sobre funcionalidad → hoy no alcanza con que el producto "haga lo que tiene que hacer" (enfoque en tareas/funcionalidad). El cliente evalúa la **experiencia completa** (UX/UI), cómo se siente usarlo, la conexión emocional. Un producto que funciona perfecto pero es frustrante de usar puede fracasar igual que uno que no funciona.

User stories y MVP es lo que se evalúa en el parcial en la parte práctica